IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Redes convolucionales: cómo «ve» una red

Imágenes como números, convolución, pooling y redes convolucionales (CNN)

Sprint 11 · S11_01 · cuaderno: 40 min en clase (ejercicios 1 y 2)

LA IDEA · SECCIÓN 1

Una imagen es una tabla de números

0	0	0	2	16	16	2	0
0	0	0	4	16	16	2	0
0	1	4	12	16	12	0	0
0	7	16	16	16	12	0	0
0	0	3	10	16	14	0	0
0	0	0	8	16	12	0	0
0	0	0	6	16	16	2	0
0	0	0	2	12	15	4	0

Un dígito real de 8 × 8 píxeles: 0 = blanco, 16 = tinta

Escala de grises

Una tabla **(alto, ancho)**: un dígito de 8 × 8 son **64 números**.

Color: 3 canales

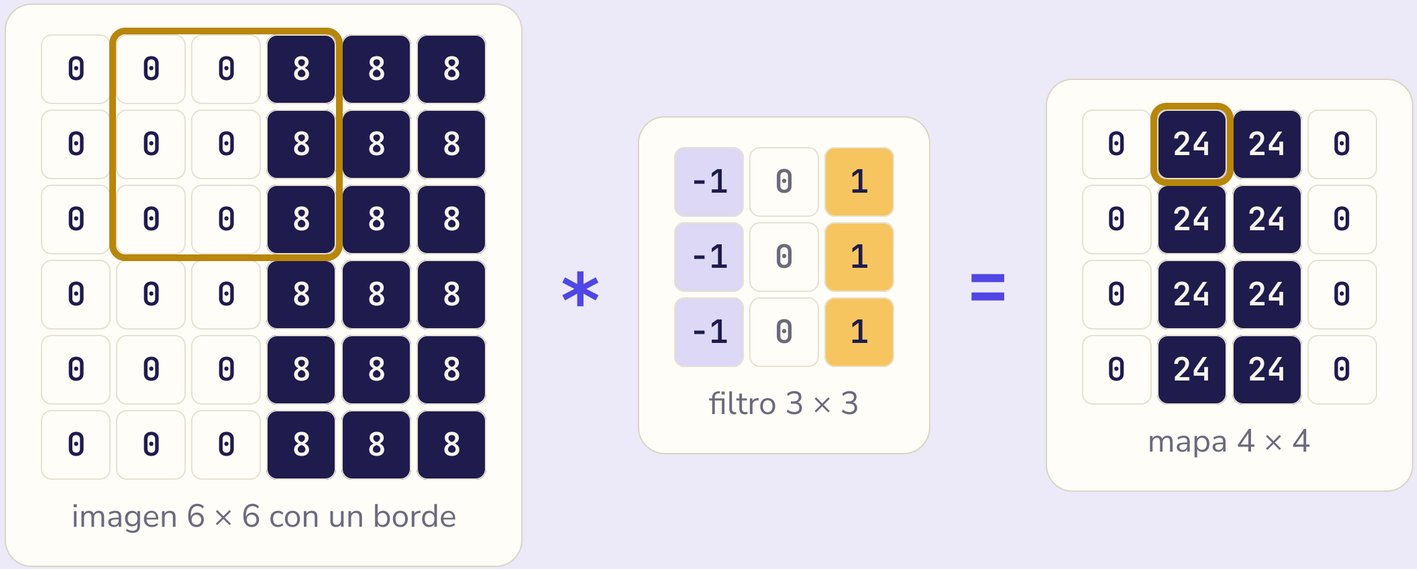
Una tabla por color (rojo, verde, azul): **(alto, ancho, 3)**. Una foto de 32 × 32 son **3 072** números.

Un lote de imágenes

Añade una dimensión delante: **(n, alto, ancho, 3)**.

LA IDEA · SECCIÓN 2

La convolución: un filtro que se desliza



Una posición, un número

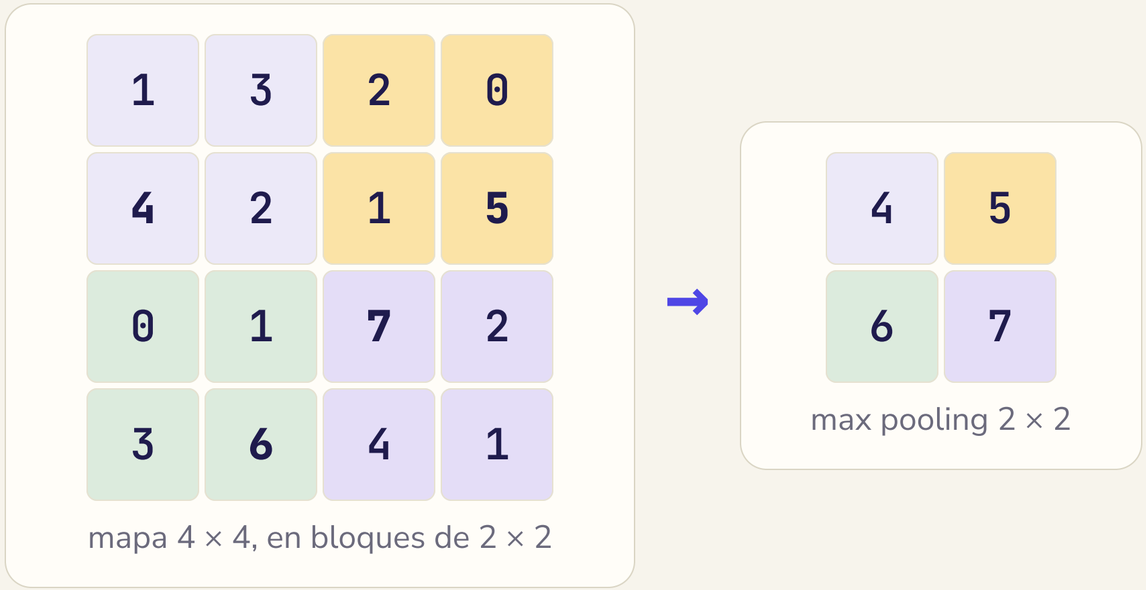
En el marco: cada fila da 0·(−1) + 0·0 + 8·1 = 8, y las tres filas suman 8 + 8 + 8 = **24**. El filtro se desliza y repite la cuenta.

El mapa dice dónde está el patrón

Valor alto (24): aquí hay un **borde vertical**. Valor 0: aquí no. Una CNN **aprende** sus propios filtros al entrenar.

LA IDEA · SECCIÓN 3

Pooling: quedarse con lo más fuerte



El máximo de cada bloque

El mapa se queda en la **mitad** de lado: menos cálculo.

Tolerancia a pequeños desplazamientos

Si el 9 cambia de casilla **dentro del bloque**, el máximo sigue siendo 9.



LA IDEA · SECCIÓN 4

Una CNN: filtros, pooling y una decisión

Capa (ejemplo de 12 × 12, 5 clases)	Salida	Parámetros
Entrada: imagen de 12 × 12 en grises	12 × 12 × 1	0
Conv2D · 8 filtros de 3 × 3	12 × 12 × 8	80
MaxPooling2D · 2 × 2	6 × 6 × 8	0
Conv2D · 16 filtros de 3 × 3	6 × 6 × 16	1 168
MaxPooling2D · 2 × 2	3 × 3 × 16	0
Flatten (aplanar)	144	0
Dense · 5 clases, softmax	5	725
Total		**1 973**

El patrón

[Conv → ReLU → Pool] × n → Flatten → Dense

De trazos a formas

Las primeras capas detectan trazos simples; las siguientes los combinan en formas cada vez más complejas.

Pocos parámetros

El mismo filtro se reutiliza en **toda** la imagen.

EJERCICIO 1 · CUADERNO (CLASE)

Preparar las imágenes para la red

Qué tengo (ejemplos propios)	Forma	Números
Una imagen en grises de 10 × 10	**(10, 10)**	100
La misma, con el canal explícito (lo que pide Keras)	**(10, 10, 1)**	100
Una foto en color de 32 × 32	**(32, 32, 3)**	3 072
Un lote de 20 fotos en color	**(20, 32, 32, 3)**	61 440

1 · Normalizar

Los píxeles van de 0 a un máximo (255 en una foto corriente). Se **divide entre ese máximo** y quedan entre 0 y 1.

2 · Entrenamiento y prueba

train_test_split con **test_size=0.25** y **stratify**: 2 000 imágenes dan 1 500 y 500, con la misma proporción de cada clase.

Si Keras protesta por la forma, mira el canal: falta una dimensión al final.

EJERCICIO 2 · CUADERNO (CLASE)

¿Cuánto mide la salida de una capa?

salida = (n − k + 2·p) ÷ s + 1

n entrada · **k** filtro · **p** relleno (padding) a cada lado · **s** paso (stride). ÷ es división entera: se descarta el resto.

n	k	p	s	Cuenta	Salida
10	5	0	1	(10 − 5 + 0) ÷ 1 + 1	**6**
10	3	0	2	(10 − 3 + 0) ÷ 2 + 1	**4**
9	3	1	2	(9 − 3 + 2) ÷ 2 + 1	**5**
28	5	2	1	(28 − 5 + 4) ÷ 1 + 1	**28**

Con paso 1 y relleno justo (**padding="same"**), la salida mide **lo mismo** que la entrada: última fila.

CNN frente a MLP: precisión y parámetros

Dígitos 8 × 8 (cuaderno)	Parámetros	Precisión en prueba
MLP (Flatten → Dense)	4 810	0,964
CNN (2 × Conv + Pooling)	6 090	0,976

Mirar dentro · Sección 5

Los filtros de la 1.ª capa son plantillas de trazos y los mapas de activación muestran **dónde** se activa cada filtro con un dígito.

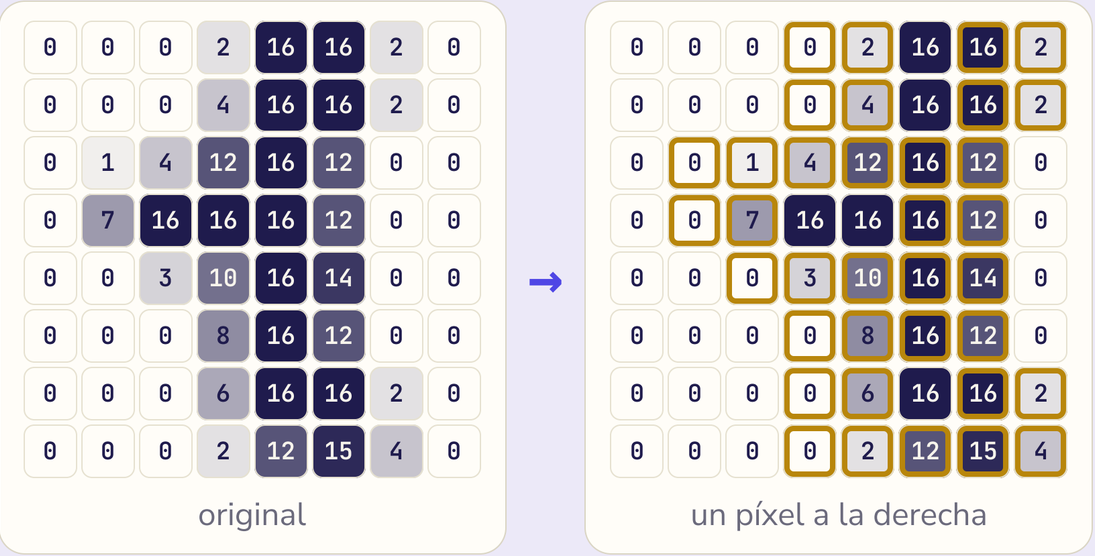
Primera capa según la foto	Valores de entrada	Dense(64)	Conv2D(16 filtros 3 × 3)
8 × 8 en grises	64	4 160	160
32 × 32 en color	3 072	196 672	448
96 × 96 en color	27 648	1 769 536	448

La ventaja de la CNN crece con el tamaño de la foto: comparte los filtros por toda la imagen.

Sección 7 (opcional, Colab): fotos reales de flores con **tf_flowers**. Necesita internet y no se ha verificado en el taller.

AUMENTO DE DATOS Y MINI-RETO · CUADERNO (CASA)

Si la imagen se mueve un píxel…



Con aro dorado, las **36 de 64** casillas que cambian de valor.

Aumento de datos

Variaciones aleatorias del entrenamiento: **RandomTranslation**, **RandomRotation**, **RandomZoom**, **RandomFlip**. **Solo al entrenar**, nunca en la prueba.

Mini-reto

Construye **cnn_aug** (la CNN con las capas de aumento **al principio**) y compara su precisión con dígitos desplazados frente a la CNN y el MLP.

Reflexión 3: ¿cuándo sería un error RandomFlip("horizontal")? Piensa en carteles con texto.

HOY HAS APRENDIDO

Así «ve» una red una imagen

Una tabla de números

Una imagen es **(alto, ancho, canales)**; se normaliza antes de entrenar

Filtro y pooling

La convolución detecta patrones con un filtro que se desliza; el pooling reduce quedándose con el máximo

CNN

Bloques de Conv y Pooling, luego Dense: filtros compartidos y aumento de datos solo al entrenar

Siguiente, hoy: S11_02 · Transfer learning y fine-tuning